

AZURE AI FOUNDRY · MULTI-AGENT REFERENCE SOLUTION

MMC Multi-Agent Manufacturing Assistant

A Magentic manager turns one plant-floor question into a plan — then dispatches a network of **10 portal-managed Foundry prompt agents** grounded in policy docs and **live Azure SQL rows**.



WHAT IT IS

One question in. A cited, ledger-grounded answer out.

A user asks a real manufacturing question — e.g. “an open PO for a brake-caliper part is flagged At Risk; which Plant 7 Line 1 PM tasks use it, and is there a backup supplier?”

A Magentic manager decomposes it into a plan, picks the right specialists from a pool of 10 agents, dispatches them in sequence or parallel, watches every output, replans when a path dead-ends, and synthesizes a final answer **with citations down to the individual SQL row.**

Every plan, hop, and replan streams live to a FastAPI + SSE trace UI — the orchestration is watchable in real time.

10



portal-managed
Foundry prompt agents

2



Azure AI Foundry
projects (plant + ent.)

283



live SQL rows across
5 change-tracked tables

Three tiers, one source of truth

Everything below derives from a single profile.yaml — agents, knowledge sources, and tools all live in one place.

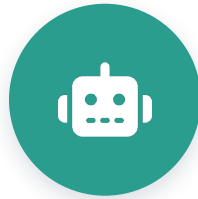


TIER 1

Orchestrator

Magentic manager

- Decomposes the question into a plan
- Dispatches agents, watches the ledger, replans
- Manager (gpt-5.4) + workers (gpt-5.4-mini) on separate deployments



TIER 2

Agents

10 Foundry prompt agents

- Portal-managed PromptAgentDefinitions — visible & editable in Foundry
- 5 plant + 5 enterprise specialists
- Foundry IQ knowledge baked in as an MCP tool



TIER 3

Knowledge

Foundry IQ + Azure SQL

- Policy docs (SOPs, OEM manuals) via doc index
- Live operational rows via IndexedSqlKnowledgeSource
- Managed-identity auth + SQL change tracking

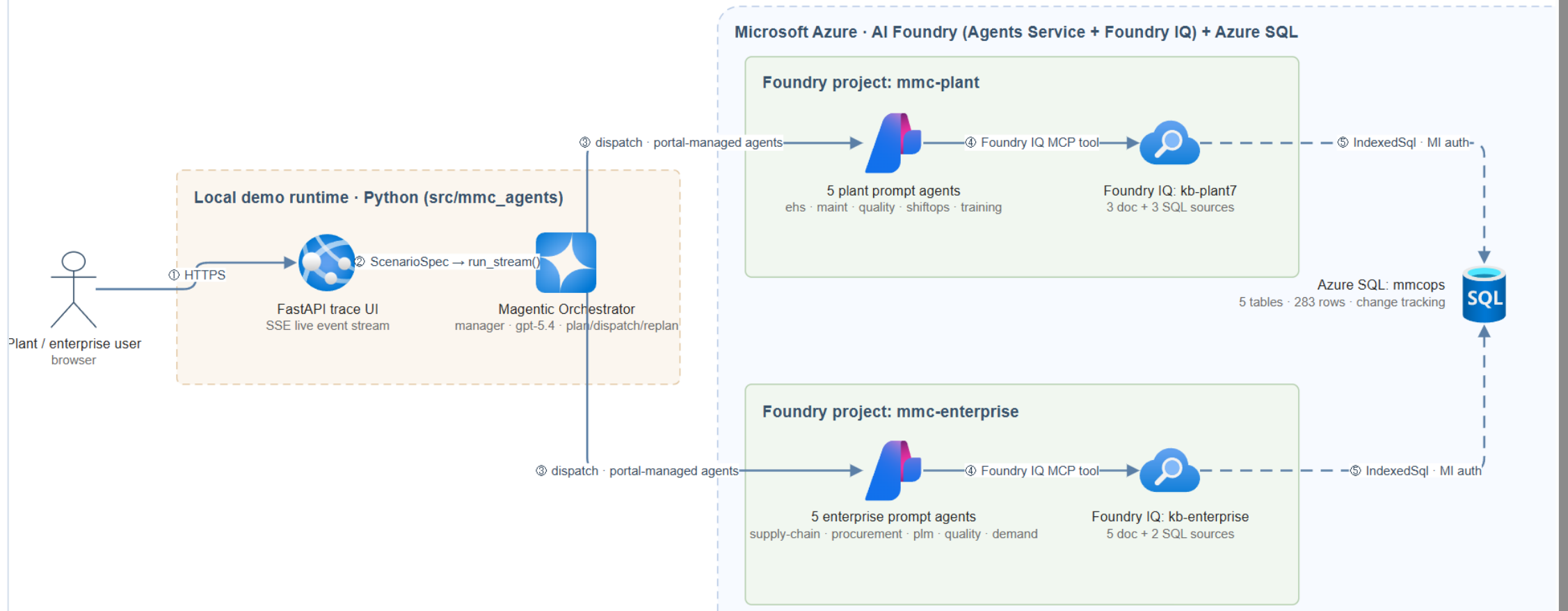
Solution architecture – Container view (C4 L2)

MMC Multi-Agent Manufacturing Assistant — Solution Architecture

Type: Container diagram (C4 L2) · as-built demo

A Magentic manager decomposes a plant-floor question and dispatches 10 portal-managed Azure AI Foundry prompt agents (5 plant + 5 enterprise), each grounded in a Foundry IQ knowledge base that blends policy docs with live Azure SQL rows; the run streams to a FastAPI + SSE trace UI.

v1.0 | Updated 2026-06-17 | Microsoft · MMC demo



Ten specialists across two Foundry projects

**mmc-plant**
Plant 7 floor operations

plant7-ehs	EHS · machine guarding · incidents
plant7-maintenance	PM schedules · work orders
plant7-quality	NCR/CAPA · QA specs
plant7-shiftops	changeover · shift handover
plant7-training	LOTO certs · competency

**mmc-enterprise**
Cross-plant enterprise functions

ent-supply-chain	suppliers · logistics · BOM
ent-procurement	POs · contracts · should-cost
ent-engineering-plm	ECOs · part numbering
ent-quality	warranty · recall thresholds
ent-demand-program	OEM programs · allocation

The orchestrator merges both pools into one participant set of up to 10 — each scenario whitelists only the agents it needs.

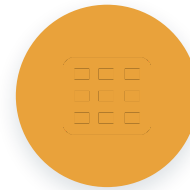
Two kinds of knowledge, deliberately split



Policy documents

`searchIndex` · `doc indexer`

- SOPs, OEM manuals, regulatory & quality docs
- Markdown / PDF pushed to Azure AI Search
- Answers the “how / why / per-procedure” questions



Operational rows

`IndexedSqlKnowledgeSource`

- Training, PM, incidents, suppliers, POs
- Indexed live from Azure SQL, PK as document key
- Returns discrete, cited rows — listed verbatim



Why split docs from rows?

When the same data lived in both, the agent paraphrased a doc chunk instead of citing the row — and replanned to `max_rounds`. Splitting row data into a SQL knowledge source made retrieval return exact, cited rows. That one change made the demo trustworthy.

From a one-agent spot-check to a ten-agent fan-out

Scenario	What it shows	Agents	Scale
<code>training_gap</code>	LOTO certs expiring — cites Training_Log rows	1	~1 hop
<code>po_status</code>	Open POs flagged At Risk — cites po_spend rows	1	~1 hop
<code>pm_check</code>	Line 1 PMs with follow-up work orders this quarter	1	~1 hop
<code>supplier_risk_pm</code>	At-risk PO → dependent PMs → backup supplier (cross-KB)	3	6–10 hops
<code>loto_cluster</code>	Cluster L1 Press near-misses → escalate to ent. quality	5–6	~15 hops
<code>brake_caliper</code>	Full fan-out: delay impact, mitigation, cost + replan	10	~30 hops

Same code, same agents, same data. Only the participant whitelist and max_rounds change — the network scales from a trivial lookup to a full multi-agent investigation.

A blueprint for trustworthy, watchable agent networks



Cited, not paraphrased

Row-level SQL grounding means every claim traces to a specific record — recall thresholds and at-risk POs you can trust.



Portal-managed & composable

Agents are Foundry PromptAgentDefinitions, visible and editable in the portal. Add an agent in profile.yaml and re-seed.



Watchable orchestration

Plan, every hop, and every replan stream live to the trace UI — the manager's reasoning is observable, not a black box.



Scales by configuration

One engine spans a 1-hop spot-check to a 10-agent fan-out. Whitelists and caps keep narrow scenarios narrow.